SYSTEM & WEB SECURITY

Complete Mid-Semester Notes, Sample Questions and Revision Sheet

Syllabus covered: Unit 1 (System Security), Unit 2 (Software Security), Unit 3 (Web Security) up to Clickjacking

Unit	Topics in these notes	Hours
1	Control hijacking, buffer/integer overflow, browser protection bypass, sandboxing, robust software, vulnerability detection, static/dynamic/concolic analysis, access control, OS security, exploitation, fuzzing	10
2	Set-UID programs, privilege separation, buffer overflow defences, return-to-libc, race condition, Dirty COW, format string, Shellshock, Heartbleed	10
3 (part)	Same-origin policy, XSS, CSRF, SQL injection, clickjacking	10 (part)

How to use: Read each unit's notes, study the highlighted exam tips and mistakes, then attempt the sample questions (2, 5 and 10 marks, case questions, MCQs) and check the model-answer outlines. Finish with the one-page revision sheet.

These notes are for learning defensive security concepts. Code shown is illustrative of vulnerable patterns and their fixes.

Contents

Unit 1: Introduction to System Security 4

1.1 Security Foundations . . . 4
1.2 Process Memory Layout and the Stack Frame . . . 4
1.3 Control Hijacking Attacks . . . 5
1.4 Defences against Control Hijacking and How They Are Bypassed . . . 7
1.5 Sandboxing and Isolation . . . 8
1.6 Tools and Techniques for Writing Robust Application Software . . . 8
1.7 Vulnerability Detection Tools and Program Analysis . . . 9
1.8 Privileges, Access Control and Operating System Security . . . 10
1.9 Exploitation Techniques and Fuzzing . . . 10

Unit 2: Software Security 12

2.1 Privileged Programs and Set-UID . . . 12
2.2 Buffer Overflow Vulnerability and Defences (Detailed) . . . 13
2.3 Return-to-libc Attack . . . 13
2.4 Race Condition Vulnerability and Attack . . . 14
2.5 Dirty COW Attack (CVE-2016-5195) . . . 14
2.6 Format String Vulnerability and Attack . . . 15
2.7 Shellshock Attack (CVE-2014-6271) . . . 16
2.8 Heartbleed Attack (CVE-2014-0160) . . . 16

Unit 3: Web Security (up to Clickjacking) 18

3.1 Web Basics and Threat Models . . . 18
3.2 Same-Origin Policy (SOP) . . . 18
3.3 Cross-Site Scripting (XSS) . . . 19
3.4 Cross-Site Request Forgery (CSRF) . . . 20
3.5 SQL Injection (SQLi) . . . 21
3.6 Clickjacking (UI Redressing) . . . 22

Sample Questions and Practice 23

Section A: Short answer (2 marks each) . . . 23
Section B: Medium answer (5 marks each) . . . 23
Section C: Long answer (10 marks each) . . . 24
Case / application questions . . . 24
Multiple choice (answers at the end of this section) . . . 25

True / False with reasons 25
Model-answer outlines for likely long questions 26

Quick Revision Sheet **27**

One-page recall table 27

Unit 1: Introduction to System Security

Syllabus: Control hijacking attacks (buffer overflow, integer overflow), bypassing browser memory protection, sandboxing and isolation, tools and techniques for writing robust application software, security vulnerability detection tools and techniques, program analysis (static, concolic and dynamic), privileges, access control and operating system security, exploitation techniques and fuzzing.

1.1 Security Foundations

Term	Meaning	Example
Vulnerability	A weakness in design, code or configuration	strcpy() into a fixed buffer with no length check
Threat	Potential cause of harm (actor or event)	Remote attacker, malware, insider
Exploit	Code/input that takes advantage of a vulnerability	Crafted packet that overflows a buffer
Attack	Actual attempt to violate a security property	Using that exploit on a live server
Risk	Likelihood x impact of a threat exploiting a vulnerability	High for internet-facing unpatched service

CIA triad and related goals

- **Confidentiality** - only authorised parties read data. **Integrity** - data/programs are not modified without authorisation. **Availability** - services usable when needed.
- **Authentication** (who are you), **Authorisation** (what may you do), **Accountability/Non-repudiation** (audit trail).
- **Memory-safety** failures (overflows, use-after-free) typically break *all three*: leaks secrets, corrupts state, crashes services.

Saltzer and Schroeder design principles (frequently asked)

Principle	Idea
Economy of mechanism	Keep design small and simple so it can be verified
Fail-safe defaults	Default is deny; access only by explicit permission
Complete mediation	Check every access to every object, every time
Open design	Security must not depend on secrecy of the design (Kerckhoffs)
Separation of privilege	Require multiple conditions/keys for sensitive actions
Least privilege	Give each subject only the rights needed for its job
Least common mechanism	Minimise shared state/mechanisms between users
Psychological acceptability	Secure way must be easy; otherwise users bypass it

Defense in depth means layering independent protections (secure code + compiler mitigations + OS protections + sandbox + monitoring), so that failure of one layer does not mean total compromise. **Attack surface** is the sum of all points where an attacker can supply input or interact (ports, APIs, files, IPC, parsers). **TCB (Trusted Computing Base)** is the set of hardware/software that must be correct for the system's security policy to hold - keep it small.

1.2 Process Memory Layout and the Stack Frame

Understanding control hijacking requires knowing how a process lays out memory and how a function call works.

Kernel space (not accessible from user mode)
Stack - local variables, saved registers, return addresses (grows DOWN)
unmapped gap / mmap region (shared libraries such as libc)
Heap - malloc/new (grows UP)
BSS - uninitialised globals/statics
Data - initialised globals/statics
Text (code) - read-only executable instructions

High addresses at top, low addresses at bottom

Stack frame of a function call (x86, 32-bit view)

Arguments of callee (pushed by caller)
Return address (pushed by CALL)
Saved frame pointer (EBP)
Saved registers / canary (if enabled)
Local variables / char buf[...]

Stack grows toward lower addresses, but array writes go toward HIGHER addresses - so overflowing a local buffer reaches the return address

- Call: caller pushes arguments, CALL pushes return address; callee pushes old EBP, sets EBP = ESP and reserves space for locals.
- Return: LEAVE restores ESP/EBP, RET pops the saved return address into EIP (the instruction pointer).
- Whoever controls the saved return address controls *where the program continues* - this is the heart of control hijacking.

1.3 Control Hijacking Attacks

> **KEY DEFINITION:** A **control hijacking attack** corrupts data that influences control flow (return address, function pointer, C++ vtable pointer, GOT entry, exception-handler record) so that execution is redirected to attacker-chosen code or to existing code used in unintended ways.

Root causes are almost always **memory-safety bugs** in C/C++: stack/heap buffer overflow, integer overflow leading to undersized buffers, format-string bugs, use-after-free and double-free, uninitialised memory, off-by-one errors, and type confusion.

(a) Stack buffer overflow

Occurs when a program writes more data into a fixed-size stack buffer than it can hold, overwriting adjacent memory, including the saved frame pointer and **return address**.

Vulnerable pattern (textbook)

```
void greet(char *input) {
    char name[16];
    strcpy(name, input);      // BUG: no bounds check; input may exceed 16 bytes
    printf("Hello %s\n", name);
}
```

- **Why it happens:** C does not check array bounds; functions like strcpy, strcat, sprintf, gets copy until a NUL byte, not until the buffer is full.
- **Effect:** (1) crash/denial of service (corrupted return address is invalid); (2) if the overwritten return address points to attacker-controlled *injected code*, arbitrary code execution; (3) if it points to *existing* code (libc function, ROP gadgets), code-reuse attack.
- **Privileged target** (root daemon or Set-UID program) turns this into privilege escalation.

Unsafe	Safer alternative	Note
gets(buf)	fgets(buf, sizeof buf, stdin)	gets removed from C11
strcpy(d, s)	snprintf(d, n, "%s", s) or strlcpy	strncpy does NOT guarantee NUL termination
strcat(d, s)	strlcat / snprintf with remaining size	Track remaining space carefully
sprintf(d, f, ...)	snprintf(d, n, f, ...)	Always pass the destination size
scanf("%s", b)	scanf("%15s", b) or fgets	Specify a field width

(b) Heap overflow

Overflowing a heap buffer overwrites adjacent heap chunks: allocator metadata (size/next pointers) or neighbouring objects that hold function pointers/vtables. Historic attacks abused the *unlink* operation in free() to get an arbitrary write; modern allocators add integrity checks (safe-unlinking, tcache checks, randomised/hardened allocators such as PartitionAlloc) but heap bugs remain exploitable, especially with *heap feng shui* (shaping heap layout).

(c) Integer overflow

Integer variables have fixed width, so arithmetic can **wrap around** (unsigned), behave undefined (signed overflow in C) or be **truncated/sign-extended** when converted between types. The integer bug itself rarely causes harm - it *defeats a size/bounds check* and leads to a memory-corruption bug.

Type	Explanation	Example
Unsigned wraparound	Result exceeds max value and wraps to small number	uint8_t x = 255; x + 1 == 0
Signed overflow	INT_MAX + 1 becomes negative (undefined behaviour)	int a = 2147483647; a + 1
Truncation	Wider type stored into narrower	size_t 70000 stored in uint16_t becomes 4464
Signedness error	Negative signed value converted to huge unsigned size_t	memcpy(dst, src, len) with len = -1

Integer-overflow patterns and fixes

```
// Case 1: multiplication overflow -> undersized allocation -> heap overflow
size_t n = get_count();                 // attacker controlled
Item *a = malloc(n * sizeof(Item));     // n * size wraps to a small number
for (size_t i = 0; i < n; i++) a[i] = read_item();   // writes far past the buffer

// Case 2: signedness bug defeats check
int len = get_len();                    // attacker supplies -1
if (len > 100) return;                  // -1 passes this check!
memcpy(buf, src, len);                  // len converted to size_t = huge value

// Fix: validate range and use checked arithmetic
if (len < 0 || len > 100) return;
if (__builtin_mul_overflow(n, sizeof(Item), &bytes)) abort();
```

- **Defences:** use unsigned size_t for sizes; check for overflow *before* the operation; checked-arithmetic builtins; calloc(n, size) (checks multiplication); compile with UBSan (-fsanitize=integer/undefined); use memory-safe languages or safe integer libraries (SafeInt).

(d) Other control-hijack vectors (know the names)

- **Use-after-free / double free:** pointer used after the object was freed; attacker reallocates that memory with controlled data (very common in browsers).
- **Format string:** attacker-controlled format string (covered in Unit 2).
- **Overwriting function pointers / vtables / GOT entries** and **SEH records** (Windows) instead of the return address.

- **Type confusion:** object treated as the wrong type, letting attacker-controlled fields be interpreted as pointers (JS engines).

1.4 Defences against Control Hijacking and How They Are Bypassed

Defence	How it works	Stops	Weakness / bypass
Stack canary (StackGuard, -fstack-protector)	Random value placed before return address; checked before return; abort if changed	Linear stack overflow that tramples return address	Info leak reveals canary; overwrite non-stack targets (function ptrs); forking servers reuse same canary, brute-forcible byte-by-byte
NX / DEP / W^X	Memory pages are writable XOR executable; stack/heap not executable	Direct execution of injected code	Code reuse: return-to-libc, ROP; JIT regions
ASLR / PIE	Randomise base of stack, heap, libraries, (PIE) executable at each run	Hard-coded addresses of libc/gadgets	Information leaks; low entropy on 32-bit (brute force); non-PIE modules; heap spraying
RELRO (full)	GOT made read-only after relocation (-Wl,-z,relro,-z,now)	GOT overwrite	Other function-pointer targets
CFI (Control-Flow Integrity)	Indirect calls/returns may only go to valid targets in the control-flow graph	Code reuse and pointer hijack	Coarse-grained CFI can be bypassed; performance cost
Shadow stack (Intel CET)	Separate protected copy of return addresses; compared on RET	Return address overwrite / ROP	Needs HW/OS support; JOP/COP attacks
FORTIFY_SOURCE	Compile-time/run-time bounds checks on libc calls (memcpy, strcpy...)	Known-size buffer overflows	Only when size is known to compiler
Bounds checking (ASan, SoftBound)	Shadow memory / metadata tracks object bounds	Overflows, UAF (detection)	2x slowdown - used in testing, not production

EXAM TIP: Examiners love the triple: **canary -> NX -> ASLR**, each with its bypass (**info leak / ROP / brute force or leak**). Also remember the point: no single defence is complete, hence defence in depth.

Code-reuse attacks (how NX is bypassed)

- **Return-to-libc:** return address is replaced with the address of an existing library function (e.g. system) so no injected code is needed (detail in Unit 2).
- **Return-Oriented Programming (ROP):** chains short instruction sequences already in the program/libraries called *gadgets*, each ending in RET. The attacker's 'payload' is mainly a list of gadget addresses on the stack; chained gadgets can compute anything (Turing-complete).
- **JOP/COP** use indirect jumps/calls instead of returns; defeat return-only protections.

Bypassing browser memory protection

Modern browsers are huge C++ code bases with JavaScript JIT engines, so memory bugs are common. Real-world browser exploits are **chains**:

- **Step 1 - memory corruption bug** in the renderer, typically use-after-free in the DOM or type confusion / out-of-bounds in the JS engine (JIT).
- **Step 2 - information leak** to defeat ASLR (and learn heap/library addresses), often by converting the bug into a read primitive.
- **Step 3 - bypass DEP/CFI** using ROP/code-reuse, abusing JIT-compiled memory, or data-only attacks that never touch control flow.
- **Step 4 - sandbox escape:** the renderer is sandboxed, so a *second* bug (kernel, IPC, GPU process) is needed to reach the OS. Chains like this are what Pwn2Own rewards.
- **Heap spraying:** fill the heap with many copies of attacker-controlled data so that a guessed address likely hits it; JIT spraying abuses predictable JIT output.

Browser-side mitigations: multi-process architecture with per-site renderer processes (Site Isolation), OS sandboxing, hardened allocators (PartitionAlloc, MiraclePtr to neutralise UAF), CFI, W^X for JIT pages, constant blinding in JIT, removal/limiting of high-resolution timers (Spectre), safer languages (Rust components) and continuous fuzzing.

1.5 Sandboxing and Isolation

> **KEY DEFINITION:** **Sandboxing** runs untrusted or high-risk code in a restricted environment so that, even if compromised, it can access only a limited set of resources. It embodies *least privilege* and the assumption *'the code will eventually be exploited - limit the damage'*.

Mechanism	How it isolates	Typical use
Process / virtual-memory isolation	Separate address spaces, kernel-mediated IPC	All modern OS
chroot / jail	Changes apparent root directory (weak, root can escape)	Legacy daemons, FreeBSD jails
Namespaces + cgroups	Private view of PIDs, network, mounts, users; resource limits	Containers (Docker)
seccomp-bpf	Filters allowed system calls per process	Chrome, Firefox, systemd, Docker
MAC frameworks	SELinux / AppArmor policy confines programs	Servers, Android (SELinux)
Software Fault Isolation (SFI)	Inserted checks keep memory accesses in a region (Native Client)	Browser plugins
Language-based	Type-safe VM: JVM, WebAssembly, JavaScript engine	Applets, web apps
Virtual machines / hypervisor	Hardware-enforced isolation of complete OS	Cloud, Qubes
Hardware enclaves (SGX, TrustZone)	Protect code/data even from the OS	Confidential computing

Case study: Chrome's sandbox architecture

- **Browser process** (privileged) manages UI, network, disk. **Renderer processes** (one per site) parse HTML/JS and are heavily sandboxed - no direct file or network access.
- Renderers talk to the browser via IPC (Mojo) and must *ask* for resources; the broker validates each request (complete mediation).
- An attacker needs a renderer bug **plus** a sandbox-escape bug. Site Isolation also ensures a compromised renderer cannot hold data of other sites.

Virtual machine	Container
Virtualises hardware; each VM has its own kernel	Shares host kernel; isolates with namespaces/cgroups
Strong isolation, heavier (GBs, slower boot)	Lightweight, fast, but a kernel bug can affect all containers

Limitations: sandbox escapes via kernel vulnerabilities, misconfigured permissions, shared resources, and micro-architectural side channels (Spectre/Meltdown).

1.6 Tools and Techniques for Writing Robust Application Software

- **Secure design first:** threat modelling, minimise attack surface, privilege separation, fail-safe defaults.
- **Secure coding standards** (CERT C, MISRA), allow-list **input validation** (length, type, range, format), and output encoding.
- **Prefer memory-safe languages** (Rust, Go, Java, C#, Python) for new code; in C/C++ use std::string/vector, smart pointers, bounds-checked APIs, span types.
- **Compiler and linker hardening:**

Typical hardened build

```
gcc -O2 -Wall -Wextra -Wformat -Wformat-security \
    -fstack-protector-strong -D_FORTIFY_SOURCE=2 \
    -fPIE -pie -Wl,-z,relro,-z,now -Wl,-z,noexecstack prog.c -o prog
```

- **Process:** code review, pair review of security-critical code, unit tests and regression tests for past vulnerabilities, continuous integration with static analysis, sanitizers and fuzzing.
- **Defensive runtime practices:** drop privileges early, check every return value, handle errors safely (fail closed), avoid secrets in logs, clear sensitive memory, keep dependencies patched (SCA tools).
- **Secure SDLC:** requirements, design/threat model, implementation, verification, release, response. (Microsoft SDL, OWASP SAMM.)

1.7 Vulnerability Detection Tools and Program Analysis

Program analysis automatically reasons about program behaviour to find bugs. Three families: **static** (without running), **dynamic** (by running), **concolic** (hybrid of concrete and symbolic execution).

Static analysis

- Analyses source/binary code *without executing* it. Techniques: lexical scanning (grep-like: Flawfinder, RATS), type/flow analysis, **data-flow and taint analysis** (track untrusted input to dangerous sinks), **abstract interpretation**, model checking, symbolic reasoning.
- **Tools:** Coverity, CodeQL, Clang Static Analyzer, Cppcheck, Semgrep, SpotBugs, SonarQube.
- **Pros:** covers all paths including rarely executed ones; early in development; no test input needed. **Cons: false positives** (approximation) or false negatives; by Rice's theorem exact analysis of non-trivial properties is undecidable, so tools trade off soundness vs completeness.

Dynamic analysis

- Observes the program *while running* on real inputs, often with instrumentation.
- **Tools:** Valgrind Memcheck (invalid reads/writes, leaks, uninitialised use), **AddressSanitizer (ASan)** (heap/stack/global overflow, UAF - compile with -fsanitize=address), MSan, TSan (data races), UBSan (undefined behaviour), Dr. Memory, taint trackers, debuggers (gdb).
- **Pros:** precise (real executions, few false positives), finds run-time behaviour. **Cons:** only finds bugs on paths actually executed; overhead; needs good test inputs.

Concolic (dynamic symbolic) execution

KEY DEFINITION: Concolic = CONCrete + symbOLIC. Run the program on a concrete input while simultaneously tracking *symbolic* expressions for inputs. At each branch record the **path constraint**; negate one branch condition and ask an SMT/constraint solver for a new input that takes the other path.

Worked example (good for a 5-mark answer)

```
void f(int x, int y) {
    if (x > 10) {
        if (y == 2 * x) abort();      // hidden bug
    }
}
Run 1: x=0,y=0    -> path constraint: NOT(x>10)
  negate last    -> x>10                  -> solver gives x=11,y=0
Run 2: x=11,y=0   -> constraints: x>10 AND y != 2x
  negate last    -> x>10 AND y == 2x      -> solver gives x=11,y=22
Run 3: x=11,y=22  -> abort() reached: bug found (random testing would almost never find it)
```

- **Tools:** DART, CUTE, SAGE (Microsoft), KLEE, angr, S2E, Mayhem. **Challenges:** path explosion (loops, many branches), hard constraints (hashes, crypto), modelling environment/system calls, pointer aliasing. Often combined with fuzzing (hybrid fuzzing, e.g. Driller).

Aspect	Static	Dynamic	Concolic
Runs program?	No	Yes (concrete inputs)	Yes + symbolic tracking
Path coverage	All paths (approximated)	Only executed paths	Systematically explores new paths
False positives	Many	Few	Very few (inputs are real)
Main cost	Imprecision	Needs inputs, overhead	Path explosion, solver time
Examples	Coverity, CodeQL	Valgrind, ASan	KLEE, SAGE, angr

1.8 Privileges, Access Control and Operating System Security

An access control system decides whether a **subject** (user/process) may perform an **operation** on an **object** (file, device, socket). It is enforced by a **reference monitor** which must be *always invoked* (complete mediation), *tamper-proof* and *small enough to verify*.

- **Access control matrix:** rows = subjects, columns = objects, cells = rights. Stored by column = **ACL** (per object list of who may do what); stored by row = **capability list** (each subject holds unforgeable tokens).
- **DAC** (owner decides, Unix permissions), **MAC** (system-wide policy, labels, SELinux; Bell-LaPadula: no read up, no write down for confidentiality; Biba for integrity), **RBAC** (permissions attached to roles), **ABAC** (attributes/policies).

Unix/Linux privilege model

- Each process has real/effective UID and GID; UID 0 (root) bypasses permission checks. Files have owner/group/other rwx bits; directories need x to traverse.
- **setuid/setgid bit:** program runs with the file owner's privilege (passwd, sudo). **Sticky bit** on /tmp prevents users deleting others' files.
- **Linux capabilities** split root's power (CAP_NET_BIND_SERVICE, CAP_SYS_ADMIN...) for finer least privilege. **SELinux/AppArmor** confine even root-owned processes.
- **Privilege escalation:** vertical (user to root) vs horizontal (user to another user).
- **OS hardening:** kernel/user separation by protection rings and system-call interface; KASLR, SMEP/SMAP; secure boot; patching; removing unused services; auditing/logging; separate accounts; full-disk encryption.

1.9 Exploitation Techniques and Fuzzing

Anatomy of exploit development (defender's understanding)

Stage	What happens
1 Discovery	Find a bug (fuzzing, code audit, patch diffing)
2 Root-cause analysis	Understand what is corrupted and what attacker controls; triage exploitability
3 Build primitives	Turn the bug into read/write/execute primitives (arbitrary read, write-what-where)
4 Bypass mitigations	Info leak for ASLR/canary, ROP for NX, data-only attacks for CFI
5 Payload and delivery	Achieve the goal (code execution, privilege escalation), then escape sandbox if needed

- **Families of technique:** code injection, code reuse (ret2libc, ROP, JOP), data-only attacks, heap feng shui/spraying, information leaks, race-condition exploitation, type confusion.

Fuzzing

> KEY DEFINITION: **Fuzzing** is automated testing that feeds large numbers of malformed, unexpected or random inputs to a program and monitors for crashes, hangs, assertion failures or sanitizer reports.

Criterion	Types
Input generation	**Mutation-based** (modify valid seeds: flip bits, splice) vs **generation-based** (from grammar/protocol model)

Criterion	Types
Knowledge of program	**Black-box** (no feedback), **grey-box** (coverage feedback, e.g. AFL), **white-box** (symbolic/concolic, e.g. SAGE)
Target	File-format, network-protocol, API/library, kernel (syzkaller), browser/JS engines, web (parameter) fuzzing

Coverage-guided greybox fuzzing (AFL, libFuzzer, honggfuzz): instrument the binary to measure edge coverage; keep any input that reaches new code in the corpus; mutate corpus entries; repeat. Combined with sanitizers (ASan/UBSan) to detect silent memory errors. Found crashes are *triaged*, de-duplicated and minimised.

Minimal fuzzing harness and command

```
clang -g -fsanitize=address,fuzzer harness.c -o fuzz_target     # libFuzzer style
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    parse_message(data, size);      // function under test
    return 0;
}
# AFL++ style:  afl-fuzz -i seeds/ -o findings/ -- ./target @@
```

- **Limitations:** hard-to-satisfy checks (magic numbers, checksums), deep stateful protocols, slow targets. Mitigated by dictionaries, structure-aware/grammar fuzzing, and hybrid concolic fuzzing.
- **Defenders use fuzzing too:** Google OSS-Fuzz, CI fuzzing, and it is the main way modern memory bugs are found before attackers find them.

Unit 2: Software Security

Syllabus: Vulnerabilities, attacks and countermeasures: privileged programs (Set-UID programs) and vulnerabilities, privilege separation, buffer overflow vulnerability and defences, return-to-libc attack, race condition vulnerability and attack, Dirty COW attack, format string vulnerability and attack, Shellshock attack, Heartbleed attack. (The phrase 'Interactivity, Annotation and Arrangement' at the end of the unit looks like a formatting artefact in the syllabus; confirm with your instructor.)

2.1 Privileged Programs and Set-UID

Some tasks need more privilege than a normal user has (e.g. changing your own password requires writing /etc/shadow, which only root may modify). Unix solves this with **Set-UID (SUID) programs**: a program whose file has the setuid bit runs with the **effective UID of its owner**, not of the user who launched it.

Recognising a Set-UID program

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root ... /usr/bin/passwd      # 's' = setuid, owner is root
$ chmod 4755 prog        # set the setuid bit (octal 4xxx)
```

ID	Meaning
Real UID (RUID)	The user who actually started the process
Effective UID (EUID)	The identity used for permission checks. For Set-UID root programs EUID = 0
Saved Set-UID	Lets the process drop privilege temporarily and later regain it

Why Set-UID programs are dangerous

The program runs with **more privilege than the user** but is influenced by the user's input, environment and files. A single bug gives the attacker the owner's (often root's) privileges. Attack surfaces:

Surface	Problem	Countermeasure
Explicit input (args, files, stdin)	Buffer overflow, format string, injection	Validate input; safe APIs
Environment variables	PATH (attacker's 'ls' runs first), LD_PRELOAD / LD_LIBRARY_PATH (attacker's shared library loaded)	Sanitise or clear environment; use absolute paths; loader ignores LD_* for Set-UID
system() calls	system('cmd') invokes a shell and obeys PATH, IFS, metacharacters	Use execve()/execv() with argument array, absolute path
Capability leaking	Privileged file descriptor still open after dropping privilege	Close descriptors before dropping privilege; O_CLOEXEC
Race conditions	TOCTTOU on files (2.4)	Atomic operations; do not use access()+open()
Dynamic linking	Library search path hijack	Static linking, RPATH care, secure loader mode

> **REMEMBER:** In Set-UID programs the **user chooses the environment**; the program must treat it as untrusted. Favourite exam line: 'Privileged programs must treat ALL inputs, including environment variables, as hostile.'

Privilege separation and least privilege

- **Privilege separation:** split a program into a small privileged part (monitor) and a large unprivileged part that does the risky work (parsing, network I/O). Example: **OpenSSH** - the pre-authentication network code runs as an unprivileged, chrooted child; only a small monitor keeps root rights and answers narrow IPC requests.
- **Drop privileges** as soon as they are no longer needed (seteuid(getuid()), setresuid) and verify that dropping succeeded. Bind to port 80 as root, then drop to an ordinary user.

- Use Linux capabilities instead of full root; split one big Set-UID binary into several small single-purpose ones.
- **Benefit:** a bug in the big unprivileged part yields little; the TCB shrinks to the small monitor.

2.2 Buffer Overflow Vulnerability and Defences (Detailed)

(Concept introduced in Unit 1; here we focus on the lab-style, exam-style treatment, particularly with Set-UID programs.)

Conditions for a stack-overflow attack

- 1. The program copies **attacker-controlled data** into a fixed-size buffer **without bounds check**.
- 2. The buffer is on the stack (or another place where overwriting reaches control data).
- 3. The attacker can make the overwritten control data point somewhere useful (injected code, existing code) - this needs knowledge of addresses, which ASLR obscures.
- If the program is Set-UID root, success gives a **root shell** - privilege escalation.

Countermeasures - summary table

Layer	Countermeasure	Notes
Developer	Safe functions, length checks, safer languages	Fix the root cause
Compiler	StackGuard canary (-fstack-protector), FORTIFY_SOURCE, warnings	Detects return-address overwrite before RET
OS	ASLR (kernel.randomize_va_space), non-executable stack (NX)	Makes address guess and injected code fail
Hardware	NX bit, shadow stack (CET), pointer authentication (ARM PAC)	Enforced in silicon
Shell	dash/bash drop privileges when EUID differs from RUID	Historic Set-UID shell countermeasure
Testing	ASan, fuzzing, static analysis	Find bugs before shipping

- **Canary variants:** terminator canary (contains NUL/CR/LF so string functions cannot copy past it), random canary, random-XOR canary.
- **Address randomisation weaknesses:** entropy is low on 32-bit systems, so repeated tries can succeed against a service that restarts/forks; info leaks remove randomness completely.

2.3 Return-to-libc Attack

> **KEY DEFINITION:** **Return-to-libc (ret2libc)** is a code-reuse attack: instead of injecting new code, the attacker overwrites the return address so that the function 'returns' into an **existing library function** (such as system() in libc), with the stack prepared so that the function receives attacker-chosen arguments. It **defeats a non-executable stack**.

Why it works: NX/DEP only forbids executing data pages. libc is legitimate executable code that is already mapped into every process, and it contains powerful functions. When the vulnerable function executes RET, the CPU pops the attacker-supplied address and jumps to that library function; the next stack slots are interpreted by the callee as *its return address* and *its arguments*, following the normal calling convention.

Buffer (filled with padding)
Saved frame pointer (padding)
Return address -> address of libc function (e.g. system)
Fake 'return address' of that function (where to go afterwards, e.g. exit for a clean exit)
Argument slot -> pointer to a string already in memory

Stack after overwrite (conceptual): the attacker needs three things - where libc lives, how many bytes to the return address, and a usable argument string

- **What the attacker must know:** addresses of the library function and of a suitable argument string, and the offset from the buffer to the return address. **ASLR makes these addresses unpredictable**, so ret2libc needs an information leak or brute force on 32-bit systems.
- **Generalisation:** *Return-Oriented Programming* chains many gadgets instead of a single function; ROP works even when dangerous functions are removed from libc.
- **Countermeasures:** ASLR and PIE; stack canaries (attack still needs to overwrite the return address); CFI and shadow stacks (return must match call); removing dangerous functions or restricting them (seccomp); on Set-UID programs, shells that drop privileges; fixing the underlying overflow.

EXAM TIP: Compare in an exam: **Shellcode injection** needs executable stack; **ret2libc** needs no injected code and works with NX; **ROP** needs no library function at all, only gadgets.

2.4 Race Condition Vulnerability and Attack

KEY DEFINITION: A **race condition** exists when the outcome of a program depends on the relative timing of events (threads, processes, signals) and an attacker can influence that timing to violate a security assumption. The classic form is **TOCTTOU - Time Of Check To Time Of Use**.

In a privileged program, a check and the use of the checked resource are two separate system calls. In the gap the attacker changes what the name refers to (typically replacing a file with a **symbolic link** to a protected file). The program then uses its privileges on the wrong object.

Classic TOCTTOU pattern (defensive illustration)

```
// Set-UID root program that wants to write to a user-supplied file only if the USER may
if (access(path, W_OK) == 0) {          // CHECK: uses REAL uid - user may write 'path'?
    /* --- window: attacker re-points 'path' to a protected file (symlink) --- */
    FILE *f = fopen(path, "w");         // USE: opens with EFFECTIVE uid (root!)
    fwrite(data, 1, len, f);
}
```

The attack is a **race**: the attacker repeatedly swaps the target, so that eventually the swap lands between check and use. Because one attempt may fail, the attacker runs the swap in a loop - exploitation needs many tries but is fully automatable. Races also appear in temporary file creation (predictable names in /tmp), in multi-threaded code, and in kernels (Dirty COW).

Countermeasures

- **Atomic operations:** make check and use one indivisible step, e.g. open with O_CREAT|O_EXCL (fail if file exists), mkstemp() for temp files.
- **Use file descriptors, not names:** open the file once, then fstat()/fchown() on the descriptor, which cannot be redirected.
- **Drop privilege:** temporarily set EUID = RUID so the normal OS check applies to the open() itself (principle of least privilege) - removes need for access().
- **Do not follow symlinks:** O_NOFOLLOW; sticky bit on /tmp plus Linux *protected_symlinks* (fs.protected_symlinks=1) blocks symlink tricks in world-writable sticky directories.
- Use locks carefully, use unpredictable temp names, and avoid privileged operations on user-controlled paths.

COMMON MISTAKE: Students often write 'use access() to check first'. That **is** the vulnerable design. The fix is to eliminate the gap, not to shorten it.

2.5 Dirty COW Attack (CVE-2016-5195)

Dirty COW is a Linux kernel **race condition in the copy-on-write (COW) mechanism** that existed for about nine years (Linux 2.6.22 to 4.8.3) and allowed an unprivileged local user to write to **read-only** memory mappings, leading to **privilege escalation**.

Background: copy-on-write

When a process maps a file privately (mmap with MAP_PRIVATE), the kernel initially shares the physical page with the page cache. On the first write, the kernel makes a **private copy** of the page (copy on write) and applies the write to the copy, so the original file is never modified. COW is also used by fork().

The bug

- Writing through a read-only private mapping (e.g. via /proc/self/mem) takes several kernel steps: find page, notice it must be COW-copied, make the copy, retry the write on the copy.
- These steps were not atomic. A second thread calling **madvise(MADV_DONTNEED)** can throw away the private copy *between* the steps.
- On retry the kernel looks the page up again and, because of the flawed logic, may now use the **original page-cache page** with write permission. The write therefore modifies the **underlying read-only file** in memory (and later on disk).
- Two threads racing in a loop (one writing, one calling madvise) eventually hit the window.

Thread A: write to private read-only mapping (triggers COW)
Kernel: allocate private copy, about to retry the write
Thread B: madvise(MADV_DONTNEED) discards the private copy
Kernel retries -> resolves to ORIGINAL page, now writable
Result: read-only file (e.g. SUID binary / /etc/passwd) modified by unprivileged user

- **Impact:** modify any file the user can read, e.g. a Set-UID program or /etc/passwd, to gain root. Even affected Android devices. Container escape possible since kernel is shared.
- **Fix:** kernel patch in October 2016 changing how the COW condition is tracked (a dedicated FOLL_COW flag instead of relying on the page's write permission), making the operation consistent. **Lesson:** kernel races are dangerous; bug lived years undetected; patch quickly and use live-patching.
- **Mitigation before patch:** upgrade kernel; restrict local untrusted users; some systems blocked /proc/self/mem writes or madvise via seccomp/SystemTap.

2.6 Format String Vulnerability and Attack

Functions of the printf family take a **format string** that controls how many arguments they read from the stack and how they are interpreted. The vulnerability arises when the **format string itself is controlled by the attacker**.

Vulnerable vs safe

```
printf(user_input);          // VULNERABLE: user_input is used as the format string
printf("%s", user_input);    // SAFE: format is a constant, input is only data
```

Specifier	Behaviour	Abuse when attacker controls the format
%x / %p	Print next stack word in hex	**Leak** stack contents: canaries, pointers, addresses (defeats ASLR)
%s	Treat next stack word as pointer to string	Read memory at an address; crash if invalid (denial of service)
%n	**Write** the number of characters printed so far to the address in the next stack word	**Arbitrary memory write** (overwrite return address, GOT, variables)
%N$x, %.Nx	Direct parameter access, padding width	Reach specific stack slot; control the written value precisely

Why printf reads from the stack: printf is variadic - it has no idea how many arguments were passed. It simply fetches one stack word per conversion specifier. With more specifiers than arguments, it walks past the real arguments into the caller's frame. With %n the attacker turns a *read* function into a *write* primitive.

- **Consequences:** crash (DoS), information disclosure (memory leak), arbitrary write, code execution, privilege escalation in Set-UID programs.

- **Countermeasures:** always use a constant format string; compile with -Wformat -Wformat-security (-Werror=format-security); _FORTIFY_SOURCE=2 blocks %n in writable format strings; static analysis and code review; avoid %n; modern libcs, languages with typed formatting.
- **Logging pitfalls:** the same bug in syslog(), fprintf(), snprintf(), err(), wrapper functions - any variadic function that accepts a format.

2.7 Shellshock Attack (CVE-2014-6271)

Shellshock (September 2014) is a vulnerability in **GNU Bash** that allowed remote/local attackers to **execute arbitrary commands**, present in bash for about 25 years.

Background: exported shell functions

Bash can *export* a function to child shells by placing its definition in an **environment variable**. When a new bash starts, it scans its environment; if a variable's value looks like a function definition (starts with '() {'), bash parses and defines it as a function.

The bug

- The parser continued to process text **after the closing brace** of the function body. If an attacker appended extra commands after the function definition, **bash executed them immediately** while importing the environment - before any script ran and without the user ever calling the function.
- So any data that can be placed into an environment variable that is later inherited by a bash process becomes a **command-execution vector**.

Attack vector	How attacker input reaches an environment variable
CGI web servers (most famous)	HTTP headers (User-Agent, Cookie, Referer) are copied into environment variables (HTTP_USER_AGENT...), then CGI script starts bash
DHCP clients	Malicious DHCP server options passed to shell scripts
OpenSSH ForceCommand / restricted shells	Variables such as SSH_ORIGINAL_COMMAND / TERM accepted from client
Set-UID programs calling system()	Local user sets environment, privileged program spawns bash (local privilege escalation)
Mail (procmail/qmail), CUPS, etc.	Data from network copied to environment

- **Impact:** remote code execution as the web-server user, worm/botnet propagation, data theft; millions of servers and embedded devices affected.
- **Fix:** patched parser so that only the function definition is evaluated and trailing text is not executed (several follow-up CVEs: -7169, -7186...). Later bash versions import functions with a prefix (BASH_FUNC_name%%).
- **Mitigation:** update bash; do not use bash for CGI (use dash or compiled code); sanitise/strip untrusted data before it reaches the environment; WAF rules; principle of least privilege for the web user.

> **EXAM TIP:** Key line to write: **'Shellshock is a code-injection flaw in the way bash imports exported functions from the environment; the parser executed trailing commands after the function body.'**

2.8 Heartbleed Attack (CVE-2014-0160)

Heartbleed (April 2014) is a **buffer over-read** in the OpenSSL implementation of the TLS/DTLS **Heartbeat extension** (OpenSSL 1.0.1 to 1.0.1f). It let remote attackers read up to **64 KB of server (or client) memory per request** without leaving traces in normal logs.

Heartbeat protocol

Heartbeat keeps a TLS connection alive: the client sends a *payload* together with a **payload-length field**; the server must echo the same payload back. (Example: payload 'HAT', length 3 -> server replies 'HAT'.)

The bug

- OpenSSL trusted the **claimed length** in the request and copied that many bytes from the request buffer into the response, **without checking** that it matched the actual payload size.
- If the client sends a 1-byte payload but claims length 65535, the server returns the 1 byte **plus ~64 KB of adjacent heap memory**.
- That memory may contain **private keys, session cookies, passwords, user data** from other connections. The attacker repeats the request to sample memory repeatedly.

Client -> Server: Heartbeat request { length = 65535, payload = 'A' (1 byte) }
Server (vulnerable): memcpy(response, payload, length) - no check length <= actual size
Server -> Client: 'A' + 65534 bytes of adjacent memory (keys, cookies, credentials)

- **Root cause:** missing bounds check (classic trust-the-input error) in C; memory-unsafe language; custom allocator that kept freed memory around and disabled mitigations.
- **Fix:** OpenSSL 1.0.1g added a check that discards requests whose claimed length exceeds the record size (or disable heartbeat with -DOPENSSL_NO_HEARTBEATS).
- **Response:** patch, **revoke and reissue certificates and private keys**, force password/session reset, since past leaks cannot be undone. Shows why *perfect forward secrecy* matters.
- **Lessons:** validate all length fields; prefer memory-safe code; fund review of critical open-source infrastructure (led to Core Infrastructure Initiative and LibreSSL fork); fuzzing and sanitizers would have caught it.

Attack	Bug class	Root cause	Primary impact
Buffer overflow	Memory write past buffer	No bounds check	Code execution
Return-to-libc	Code reuse	Return address overwritten; NX bypass	Code execution / root shell
Race condition	TOCTTOU	Check and use not atomic	Privileged file access
Dirty COW	Kernel race in COW	Non-atomic page-fault handling	Write read-only files, root
Format string	Attacker-controlled format	printf(input)	Info leak, arbitrary write
Shellshock	Command injection (env)	Bash parser runs trailing code	Remote code execution
Heartbleed	Buffer over-read	Missing length check in TLS heartbeat	Memory disclosure (keys)

Unit 3: Web Security (up to Clickjacking)

Covered here: Web threat models, Same-Origin Policy, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), SQL Injection and Clickjacking. CSP, web tracking, session management, HTTPS/TLS and threat modelling are after Clickjacking in the syllabus, so they are outside this mid-sem scope; CSP is mentioned only as a defence tool.

3.1 Web Basics and Threat Models

- **HTTP is stateless;** state is added with **cookies** (session IDs). The browser automatically attaches a site's cookies to *every* request to that site - the root of CSRF.
- **DOM and JavaScript** run in the browser with the privileges of the page's origin - the root of XSS.
- **Server side:** web apps build SQL queries, shell commands and HTML from user input - the root of injection attacks.

Attacker model	Capabilities
Web attacker	Owns a malicious website (evil.com) and lures victims to visit it; cannot see victim's network traffic
Network attacker	Can read/modify traffic (open Wi-Fi); defended by HTTPS
Gadget/related-domain attacker	Can inject content into a benign site (via XSS, comments, ads)

Cookie attributes

Attribute	Effect
Secure	Sent only over HTTPS
HttpOnly	Not readable from JavaScript (limits cookie theft via XSS)
SameSite=Strict/Lax/None	Controls whether the cookie is sent on cross-site requests (primary CSRF defence)
Domain / Path / Expires	Scope and lifetime of the cookie

3.2 Same-Origin Policy (SOP)

KEY DEFINITION: An **origin** is the triple **(scheme, host, port)**. The **Same-Origin Policy** says that scripts running in one origin may not read or modify the content (DOM, responses, storage) of another origin. It is the fundamental isolation mechanism of the web.

URL compared with http://shop.com/a/page.html	Same origin?	Reason
http://shop.com/b/other.html	Yes	Only the path differs
https://shop.com/page.html	No	Different scheme (https vs http)
http://shop.com:8080/page.html	No	Different port (80 vs 8080)
http://www.shop.com/page.html	No	Different host (subdomain)
http://shop.com:80/x	Yes	80 is default port for http

What SOP restricts and what it allows

- **Blocked:** reading responses of cross-origin XMLHttpRequest/fetch; accessing DOM of cross-origin iframes/windows; reading cross-origin canvas pixels; reading cross-origin storage.
- **Allowed (embedding, not reading):** loading cross-origin images, scripts (<script src>), stylesheets, iframes, forms submitted cross-origin. **Crucially, the request is still sent (with cookies)**; SOP only stops the attacker page from *reading* the response. This is why CSRF is possible.

- **Scripts are run with the origin of the embedding page:** a script loaded from cdn.com into shop.com acts as shop.com.
- **Cookies use a different rule** (domain and path, not port/scheme), which causes weaknesses (e.g. cookies shared across subdomains).

Controlled relaxations

- **CORS (Cross-Origin Resource Sharing):** server opts in via headers (Access-Control-Allow-Origin) to let specified origins read responses; preflight OPTIONS request for non-simple requests. Misconfiguration (reflecting any Origin or '*' with credentials) breaks security.
- **postMessage:** explicit, message-based cross-origin communication; always check event.origin.
- **document.domain** (legacy relaxation, deprecated), **JSONP** (legacy, insecure - script inclusion).

COMMON MISTAKE: SOP does **not** protect against XSS (attacker code runs *inside* the victim origin), CSRF (writes are allowed) or SQL injection (server-side). Do not say 'SOP prevents CSRF'.

3.3 Cross-Site Scripting (XSS)

KEY DEFINITION: XSS is an injection attack where an attacker gets a web application to deliver **attacker-controlled script** to a victim's browser, which executes it with the privileges of the vulnerable site's origin - so SOP does not help. Root cause: untrusted data placed into HTML/JS without proper output encoding.

Vulnerable pattern (proof-of-concept level)

```
<!-- Server echoes the 'q' parameter without encoding -->
<p>Results for: <?php echo $_GET['q']; ?></p>

<!-- If q contains HTML such as  <script>alert(1)</script>, the browser runs it.
     Real attacks replace alert(1) with code that steals cookies/tokens or performs actions. -->
```

Type	Where is payload stored/processed	Example scenario
Reflected (non-persistent)	Payload is in the request (URL/form) and immediately reflected in the response; victim must click a crafted link	Search page echoing the query; phishing link
Stored (persistent)	Payload is saved on server (DB) and served to every visitor	Comment, profile, forum post, chat message; most dangerous (self-spreading worms, e.g. Samy worm)
DOM-based	Vulnerability is in client-side JS that writes untrusted data (location.hash, etc.) into the DOM using innerHTML / document.write; server may never see payload	Script reads URL fragment and inserts into page

Impact

- Session hijacking (cookie/token theft unless HttpOnly), account takeover, performing actions as the victim, keylogging, phishing overlays, defacement, redirecting to malware, reading any data the victim can see in that origin, pivoting to CSRF/internal attacks.

Defences (in order of importance)

- **1. Context-aware output encoding/escaping** of all untrusted data: HTML body (<), attribute, JavaScript, URL and CSS contexts each need different encoding. Use framework auto-escaping templates (React, Angular, Jinja2, Razor).
- **2. Avoid dangerous sinks:** prefer textContent over innerHTML, avoid eval/document.write; use Trusted Types.
- **3. Sanitise rich HTML** with a proven allow-list library (DOMPurify); never hand-written blacklists/regex (easily bypassed).
- **4. Content Security Policy (CSP):** header that restricts script sources, blocks inline scripts (script-src 'self' with nonces) - defence in depth that limits impact.

- **5. HttpOnly and SameSite cookies** reduce damage; input validation (allow-list) as additional layer; X-Content-Type-Options: nosniff; correct Content-Type/charset.

COMMON MISTAKE: Blacklist filtering (removing the word 'script') is **not** a valid defence; attackers use event handlers (onerror), other tags and encodings. The correct answer is output encoding plus CSP.

3.4 Cross-Site Request Forgery (CSRF)

KEY DEFINITION: **CSRF** tricks a victim's browser, which is logged in to a target site, into sending an **unwanted state-changing request** to that site. The browser automatically attaches the victim's cookies, so the server thinks the request is legitimate. The attacker **cannot read** the response, only cause the action.

Attack flow

1. Victim logs in to bank.com; browser stores session cookie
2. Victim (still logged in) visits attacker's page / opens malicious email or image
3. Attacker page makes the browser send a request to bank.com (e.g. an image, hidden form or auto-submitted form that performs a transfer)
4. Browser attaches bank.com cookie automatically -> bank executes the transfer

Illustration: state change via GET is especially bad

```
<!-- on attacker's site: the browser will send this GET to bank.com WITH the victim's cookies -->
<img src="http://bank.com/transfer?to=attacker&amount=1000" width="0" height="0">

<!-- For POST: hidden form with JavaScript: document.forms[0].submit() -->
```

- **Conditions needed:** (1) cookie-based (ambient) authentication, (2) victim is logged in, (3) the request has predictable parameters (attacker knows or can guess all values), (4) no unpredictable value the attacker cannot supply.

Defences

Defence	How it works	Comments
CSRF token (synchroniser token)	Server puts a random, per-session (or per-request) secret in each form/header; rejects requests without the correct value	Standard defence; attacker cannot read the token due to SOP
Double-submit cookie	Token sent both as cookie and request parameter; server compares	Stateless; needs secure implementation (signed)
SameSite cookies	Lax/Strict stop the browser from sending cookies on cross-site requests	Strong default in modern browsers; Lax allows top-level GET navigation
Origin / Referer check	Server verifies that the request came from own origin	Headers may be missing; use as additional layer
Custom request header	AJAX requests must include a header (e.g. X-Requested-With); cross-origin pages cannot add it without CORS approval	API-friendly
Re-authentication / CAPTCHA	Ask password/OTP for sensitive actions	Good for critical operations
Use proper HTTP verbs	No state change on GET	Necessary but not sufficient

	XSS	CSRF
Nature	Attacker script runs in victim origin	Attacker makes victim's browser send a forged request
Exploits	Site's trust in user input	Site's trust in the user's browser/cookies
Reads data?	Yes (full access to page, cookies, tokens)	No (blind, cannot read response)

	XSS	CSRF
Stopped by CSRF token?	No (script can read the token)	Yes
Main defences	Output encoding, CSP, sanitisation	CSRF tokens, SameSite cookies

REMEMBER: XSS defeats all CSRF defences because injected script runs in the same origin and can read tokens; fix XSS first.

3.5 SQL Injection (SQLi)

KEY DEFINITION: **SQL injection** occurs when untrusted input is concatenated into a SQL query so that the input is interpreted as **SQL code** instead of data, allowing the attacker to alter the query's logic.

Authentication bypass (standard textbook example)

```
// Vulnerable: string concatenation
query = "SELECT * FROM users WHERE name='" + name + "' AND pass='" + pw + "'";

// Input  name = admin' --    makes the query become:
SELECT * FROM users WHERE name='admin' -- ' AND pass=''       // password check commented out

// Input  name = ' OR '1'='1   makes the condition always true -> logs in as first user
```

Type	Description
In-band: error-based	Database error messages leak structure/data
In-band: UNION-based	UNION SELECT appends attacker's query results to normal output (needs matching column count)
Blind: boolean-based	No data shown; attacker infers from true/false differences in page behaviour
Blind: time-based	Attacker makes DB sleep conditionally (e.g. SLEEP()) and measures response delay
Out-of-band	Data exfiltrated through DNS/HTTP requests from DB
Second-order	Malicious input stored safely, but later used unsafely in another query

- **Impact:** authentication bypass, reading the whole database (credentials, personal data), modification/deletion (DROP TABLE), sometimes OS command execution (xp_cmdshell), file read/write, full server compromise.
- **Root cause:** mixing code and data (same as buffer overflows and XSS: injection).

Defences

Primary fix

```
// Parameterised query / prepared statement: SQL structure is fixed, input is bound as data only
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE name=? AND pass=?");
ps.setString(1, name);  ps.setString(2, pw);
ResultSet rs = ps.executeQuery();
```

- **1. Prepared statements / parameterised queries** (best). Also safe stored procedures (if they do not build dynamic SQL) and ORMs used correctly.
- **2. Allow-list input validation** (type, length, format), especially for things that cannot be parameterised (table/column names, ORDER BY).
- **3. Least privilege** for DB account (no DROP, no file/OS access); separate accounts per application.
- **4. Do not show raw DB errors;** generic error pages; logging. **5. WAF** and monitoring as extra layers (not a substitute). **6.** Escaping is a weaker last resort; hash passwords (bcrypt/argon2) so leaked data is less useful.

EXAM TIP: Answer template: definition -> vulnerable code -> malicious input and resulting query -> impact -> fix with prepared statement. That alone scores full marks for a 5-mark question.

3.6 Clickjacking (UI Redressing)

KEY DEFINITION: **Clickjacking** tricks a user into clicking something different from what they perceive. The attacker loads the target site in a **transparent (or hidden) iframe** on top of, or beneath, a decoy page, positioned so that the victim's click lands on a sensitive button of the target site while the user thinks they are clicking the decoy.

Because the user is logged in to the target site, the click is executed with the user's authority. It does **not** need any bug in the target's code, only that the site **allows itself to be framed**. Note that CSRF tokens do not help, because the click is a genuine action from the genuine page.

Attacker page (visible): 'Click here to win a prize!' [decoy button]
Target site in iframe: opacity 0, positioned exactly over the decoy button (e.g. 'Delete account', 'Transfer', 'Like', 'Allow camera')
User clicks decoy -> click actually goes to the invisible target button -> action performed as the victim

Variants

- **Likejacking / social jacking:** forcing likes, follows, shares. **Cursorjacking:** displayed cursor position differs from the real one.
- **Drag-and-drop jacking:** user drags an item and unknowingly drops data into a hidden frame. **Permission jacking:** tricking users into granting camera/microphone access.
- **Double-clickjacking / tap-jacking (mobile):** overlays on touch interfaces.

Defences

Defence	Details	Status
CSP frame-ancestors	Content-Security-Policy: frame-ancestors 'none' (or 'self' / list of trusted origins)	**Modern standard**, supersedes X-Frame-Options
X-Frame-Options	Response header: DENY (never framed) or SAMEORIGIN (only same origin)	Widely supported, legacy
Frame-busting JavaScript	if (top != self) top.location = self.location	**Weak**: can be bypassed (sandbox attribute on iframe disables scripts, onbeforeunload tricks, double framing)
SameSite cookies	Cookies not sent in cross-site framed request, so framed page appears logged-out	Reduces impact
User confirmation	Re-authenticate/confirm dialogs for sensitive actions; CAPTCHA	Breaks the attack's 'one blind click'
Browser UI protections	Permission prompts that require delay/visibility, Intersection Observer v2 checks for occlusion	Emerging

COMMON MISTAKE: Do not answer 'CSRF tokens prevent clickjacking' - the click is performed on the real page, so tokens are valid. Use **frame-ancestors / X-Frame-Options**.

Sample Questions and Practice

Questions are grouped by marks. Attempt them closed-book, then check against the notes. Model-answer outlines for the most likely long questions follow the question sets.

> **EXAM TIP:** Mid-sem strategy: (1) for every attack write **definition -> how it works -> example -> impact -> countermeasure**; (2) draw the stack/flow diagram - diagrams earn marks; (3) always name the CVE/tool/header where you can; (4) for 'compare' questions use a table.

Section A: Short answer (2 marks each)

Q1. Define control hijacking attack. Name four control data targets an attacker may overwrite.

Q2. Differentiate vulnerability, threat, exploit and risk.

Q3. Why does a stack buffer overflow overwrite the return address even though the stack grows downwards?

Q4. What is a stack canary? Name two ways it can be bypassed.

Q5. What is the difference between NX/DEP and ASLR?

Q6. Give an example of an integer overflow leading to a buffer overflow.

Q7. What is sandboxing? Give two mechanisms that implement it.

Q8. Distinguish between static and dynamic analysis in one line each.

Q9. What is a path constraint in concolic execution?

Q10. What is the difference between an ACL and a capability list?

Q11. What is coverage-guided fuzzing?

Q12. What is a Set-UID program? Why is it dangerous?

Q13. Explain the difference between real UID and effective UID.

Q14. Why is system() considered dangerous in a Set-UID program? What should be used instead?

Q15. What does TOCTTOU stand for? Give one countermeasure.

Q16. What is the role of %n in format-string attacks?

Q17. What vulnerability did Heartbleed exploit?

Q18. What is the origin in the Same-Origin Policy? Are http://a.com and https://a.com the same origin?

Q19. Why does SOP not prevent CSRF?

Q20. Name the HTTP header/directive used to prevent clickjacking.

Section B: Medium answer (5 marks each)

Q21. Explain how a stack buffer overflow works with a diagram of the stack frame. List four safe alternatives to unsafe C functions.

Q22. Explain integer overflow vulnerabilities with three examples. How can they be prevented?

Q23. Describe the countermeasures against buffer overflow (canary, NX, ASLR, CFI, shadow stack) and how each can be bypassed.

Q24. Explain Chrome's sandbox architecture and why it reduces the impact of a browser exploit. How does a browser exploit chain normally proceed?

Q25. Compare static, dynamic and concolic analysis with tools and limitations. Illustrate concolic execution with a small example.

Q26. Explain the Unix access-control model, including setuid, sticky bit and Linux capabilities.

Q27. What is fuzzing? Explain the working of a coverage-guided greybox fuzzer and its limitations.

Q28. Explain the attack surface of Set-UID programs (environment variables, system(), capability leaking) and countermeasures.

Q29. What is privilege separation? Explain with OpenSSH as example.

Q30. Explain the return-to-libc attack. Why does it defeat a non-executable stack and how does ASLR hinder it?

Q31. Explain the race-condition (TOCTTOU) vulnerability in a Set-UID program with code and describe three countermeasures.

Q32. Explain format-string vulnerabilities: specifiers %x, %s, %n, consequences and defences.

Q33. Explain the Shellshock vulnerability: mechanism, attack vectors and fix.

Q34. Explain the Heartbleed bug: protocol background, root cause, impact and remediation.

Q35. Explain Same-Origin Policy with examples. What does it allow and block?

Q36. Explain the three types of XSS with examples. List the defences.

Q37. Explain CSRF with a scenario and describe at least three defences. Why are CSRF tokens effective?

Q38. Explain SQL injection with an example query; describe prepared statements as the countermeasure.

Q39. Explain clickjacking and discuss why frame-busting scripts are insufficient.

Section C: Long answer (10 marks each)

Q40. Discuss control hijacking attacks in detail: stack/heap/integer overflows, the modern mitigation stack, and techniques to bypass each mitigation (ROP, information leaks, spraying).

Q41. 'No single mitigation stops memory-corruption attacks.' Justify using canaries, NX, ASLR, CFI and sandboxing. Include a browser case.

Q42. Describe program analysis techniques for finding vulnerabilities (static, dynamic, concolic, fuzzing). Compare and show how they complement each other.

Q43. Discuss the security of Set-UID programs: mechanism, vulnerabilities (buffer overflow, environment, race), and principles for writing secure privileged programs.

Q44. Explain Dirty COW: background of copy-on-write, the race, the impact, the patch and lessons learned.

Q45. Compare Shellshock and Heartbleed on root cause, exploitation, impact and mitigation. What general lessons for secure software do they teach?

Q46. Discuss the web threat model and explain SOP, XSS and CSRF, showing how they relate and differ. Why can XSS defeat CSRF protections?

Q47. Explain SQL injection fully: types, examples, impact and layered defences.

Q48. A web application allows users to post comments, transfer money and embed itself in third-party sites. Identify the vulnerabilities this may expose (XSS, CSRF, clickjacking, SQLi) and recommend fixes for each.

Case / application questions

Q59. Code review: The code 'char b[32]; gets(b); printf(b);' appears in a Set-UID root program. Identify every vulnerability and propose a corrected version.

Q60. Code review: 'unsigned short n = get(); char *p = malloc(n+2); memcpy(p, src, get());' - explain what can go wrong.

Q61. Design: You must run an image converter on untrusted uploaded files on your server. Design a sandbox using privilege separation, seccomp, and containers.

Q62. Scenario: A server with NX and ASLR enabled is still exploited. List three plausible reasons.

Q63. Scenario: A banking site uses session cookies and a GET-based /transfer endpoint. Demonstrate why it is vulnerable to CSRF and redesign it.

Q64. Scenario: A login query concatenates user input. Show an input that bypasses authentication and rewrite the code safely.

Q65. Scenario: Your site must be embeddable only by partner.com. What header do you set?

Multiple choice (answers at the end of this section)

1. Which defence makes injected stack code non-executable?
a) ASLR b) NX/DEP c) Canary d) RELRO

2. ROP chains together code snippets that end with:
a) jmp b) call c) ret d) nop

3. A terminator canary contains:
a) only zeros b) random bytes c) NUL, CR, LF, 0xFF d) the return address

4. Concolic execution uses:
a) only random inputs b) concrete + symbolic execution with a solver c) only static parsing d) sandboxing

5. Which is dynamic analysis?
a) CodeQL b) Coverity c) AddressSanitizer d) Flawfinder

6. In a Set-UID root program, EUID is:
a) user UID b) 0 c) 1000 d) nobody

7. The best fix for TOCTTOU in file handling is:
a) check faster b) sleep between c) make check and use atomic d) use gets()

8. The format specifier that writes to memory is:
a) %x b) %s c) %d d) %n

9. Heartbleed was caused by:
a) integer overflow in kernel b) missing length check in TLS heartbeat c) weak cipher d) SQL injection

10. Shellshock exploited:
a) Apache b) OpenSSL c) Bash environment function parsing d) glibc DNS

11. Dirty COW is a race in:
a) copy-on-write memory handling b) Bash c) TLS d) browser JIT

12. Which URL is same-origin with https://a.com/x?
a) http://a.com/x b) https://a.com:8443/y c) https://a.com/z/q d) https://b.a.com/x

13. CSRF is best prevented by:
a) output encoding b) CSRF tokens / SameSite cookies c) WAF only d) HTTPS

14. Which is NOT a type of XSS?
a) Reflected b) Stored c) DOM-based d) Time-based

15. Best defence against SQL injection:
a) blacklisting quotes b) prepared statements c) hiding errors only d) client-side validation

16. Modern header to stop clickjacking:
a) frame-ancestors b) X-Powered-By c) Referrer-Policy d) HSTS

17. Privilege separation in OpenSSH means:
a) all code runs as root b) small privileged monitor plus unprivileged child c) no root d) password-less login

18. ASan detects:
a) memory errors like overflow and use-after-free b) CSRF c) weak passwords d) DNS spoofing

> **REMEMBER: Answers:** 1-b, 2-c, 3-c, 4-b, 5-c, 6-b, 7-c, 8-d, 9-b, 10-c, 11-a, 12-c, 13-b, 14-d, 15-b, 16-a, 17-b, 18-a

True / False with reasons

Statement	Answer
Stack canaries prevent all memory-corruption attacks.	False - they only detect linear overwrites of the return address; leaks and non-stack targets bypass them.

Statement	Answer
ASLR alone is sufficient to stop ROP.	False - info leaks or brute force on 32-bit break it.
strncpy always NUL-terminates.	False - it does not if the source is at least n bytes long.
SOP prevents a page from sending a request to another origin.	False - it prevents *reading* the response; requests are still sent.
CSRF tokens protect against XSS.	False - XSS code can read the token.
Frame-busting JS is the recommended clickjacking fix.	False - use frame-ancestors / X-Frame-Options; JS can be bypassed.
Escaping quotes is the best SQLi defence.	False - parameterised queries are.
Static analysis never reports false positives.	False - over-approximation causes false positives.

Model-answer outlines for likely long questions

A. Buffer-overflow attack, defences, bypasses

- Memory layout and stack frame diagram; vulnerable strcpy code; overflow reaches saved EBP/return address.
- Consequence: crash, injected-code execution, code reuse; worse with Set-UID/root daemons.
- Defences: safe coding, canary, NX, ASLR/PIE, RELRO, CFI, shadow stack, FORTIFY, ASan for testing.
- Bypasses: info leak vs canary/ASLR, ret2libc/ROP vs NX, brute force vs 32-bit ASLR, heap spray; conclusion: defence in depth plus memory-safe languages.

B. Concolic vs static vs dynamic vs fuzzing

- Define each; give tools; give the f(x,y) example with path constraints; comparison table; limitations (false positives, coverage, path explosion); hybrid approach (fuzzer finds breadth, concolic solves hard branches).

C. Set-UID security

- Mechanism (RUID, EUID, setuid bit, passwd example); attack surface list (input, environment, system(), capability leak, race); countermeasures: least privilege, sanitise environment, execve, close descriptors, drop privileges, privilege separation.

D. Shellshock vs Heartbleed

	Shellshock	Heartbleed
Software / CVE	GNU Bash, CVE-2014-6271	OpenSSL heartbeat, CVE-2014-0160
Class	Command injection via environment	Buffer over-read, info disclosure
Root cause	Parser executed trailing code after function body	Trusted attacker-supplied length, no bounds check
Vector	CGI headers, DHCP, SSH, Set-UID	Malformed TLS heartbeat request
Impact	Remote code execution	Leak of keys, sessions, passwords (up to 64KB per request)
Fix	Patch parser; function prefix	Bounds check (1.0.1g); rotate keys and certs
Lesson	Untrusted data in environment; old code	Validate lengths; memory-safe code; review critical OSS

E. Web trio: SOP, XSS, CSRF (+ SQLi, clickjacking)

- Threat model; SOP with origin table; XSS types and encoding/CSP defences; CSRF flow and tokens/SameSite; comparison table XSS vs CSRF; SQLi prepared statements; clickjacking frame-ancestors.
- Key insight to state: SOP stops *reading* not *sending*; XSS breaks SOP's assumption; CSRF abuses ambient cookies; clickjacking abuses UI; SQLi abuses server-side code/data mixing.

Quick Revision Sheet

One-page recall table

Topic	Remember
Stack overflow	Overwrites return address; fix: bounds checks, canary, NX, ASLR, CFI, shadow stack
Integer overflow	Wraparound/truncation/sign errors defeat size checks; check before arithmetic; calloc; UBSan
ROP	Gadgets ending in ret; bypasses NX; stopped by CFI/shadow stack/ASLR+no leak
Browser exploit chain	Renderer bug -> info leak -> code reuse -> sandbox escape (second bug)
Sandbox	chroot, namespaces/cgroups, seccomp, SFI, VMs, SELinux; Chrome renderer per site
Static / dynamic / concolic	No run / run / concrete+symbolic with SMT solver; Coverity / ASan, Valgrind / KLEE, SAGE
Access control	ACL = column, capability = row; DAC, MAC, RBAC; Bell-LaPadula no read up/no write down
Fuzzing	Mutation vs generation; black/grey/white; AFL, libFuzzer; corpus + coverage + sanitizers
Set-UID	EUID of owner; attack env vars (PATH, LD_PRELOAD), system(), capability leak; use execve
Ret2libc	Return into libc function (system) with crafted arguments; NX ineffective; ASLR hinders
Race / TOCTTOU	Check then use; symlink swap; use O_EXCL, fstat on fd, drop privileges, protected_symlinks
Dirty COW	CVE-2016-5195; COW race + madvise(DONTNEED); write read-only file; kernel 2.6.22-4.8.3
Format string	printf(user); %x leak, %s read, %n write; fix printf("%s", s)
Shellshock	CVE-2014-6271; bash runs code after function definition in env variable; CGI vector
Heartbleed	CVE-2014-0160; OpenSSL heartbeat over-read 64KB; fix 1.0.1g; revoke keys
SOP	Origin = scheme+host+port; blocks reading cross-origin responses; requests still sent
XSS	Reflected/stored/DOM; output-encode by context, CSP, HttpOnly, DOMPurify
CSRF	Ambient cookies; fix: tokens, SameSite, Origin check; can't read response
SQLi	' OR '1'='1; in-band/blind/out-of-band; prepared statements, least privilege
Clickjacking	Invisible iframe overlay; fix: CSP frame-ancestors, X-Frame-Options DENY/SAMEORIGIN

EXAM TIP: Last-day checklist: redraw the stack frame, ret2libc layout and Heartbleed diagram from memory; write the f(x,y) concolic example; list defences for each of the eight attacks; rehearse the XSS vs CSRF table. Good luck!